

In **Java 8**, there are **two main categories of interfaces**:

1. **Normal (Regular) Interface**
2. **Functional (SAM - Single Abstract Method) Interface**

Java 8 also introduced **default methods** and **static methods**, which greatly enhanced interfaces.

1. Normal (Regular) Interface

Definition

A **Normal Interface** is an interface that can contain **multiple abstract methods**.

It is used to define a contract that implementing classes must follow.

Example

```
interface Animal {  
    void eat();  
    void sleep();  
    void sound();  
}  
  
class Dog implements Animal {  
    @Override  
    public void eat() {  
        System.out.println("Dog eats.");  
    }  
  
    @Override  
    public void sleep() {  
        System.out.println("Dog sleeps.");  
    }  
  
    @Override  
    public void sound() {  
        System.out.println("Dog barks.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
Animal dog = new Dog();

dog.eat();
dog.sleep();
dog.sound();
}
}
```

Output

Dog eats.
Dog sleeps.
Dog barks.

Characteristics

- Can contain multiple abstract methods.
- Supports multiple inheritance.
- Cannot be instantiated.
- Variables are automatically:
 - public
 - static
 - final
- Methods are automatically:
 - public
 - abstract (unless they are default or static in Java 8).

2. Functional Interface (SAM Interface)

Definition

A **Functional Interface** contains **exactly one abstract method**.

It can have any number of:

- default methods
- static methods

Java 8 introduced Functional Interfaces to support **Lambda Expressions** and **Method References**.

Example

```
@FunctionalInterface
interface Calculator {
```

```

    int add(int a, int b);
}

public class Main {

    public static void main(String[] args) {

        Calculator calculator = (a, b) -> a + b;

        System.out.println(calculator.add(10, 20));
    }
}

```

Output

30

Another Example

Without Lambda

```

interface Greeting {
    void sayHello();
}

class Hello implements Greeting {

    @Override
    public void sayHello() {
        System.out.println("Hello Java");
    }
}

public class Main {

    public static void main(String[] args) {

        Greeting g = new Hello();

        g.sayHello();
    }
}

```

Using Lambda

```

Greeting g = () -> System.out.println("Hello Java");

g.sayHello();

```

Why Functional Interfaces?

They enable concise programming with:

- Lambda Expressions
 - Method References
 - Stream API
 - Functional Programming
-

Java 8 Features Added to Interfaces

1. Default Methods

A default method provides an implementation inside an interface.

Example

```
interface Vehicle {  
  
    void start();  
  
    default void stop() {  
        System.out.println("Vehicle stopped");  
    }  
}  
  
class Car implements Vehicle {  
  
    @Override  
    public void start() {  
        System.out.println("Car started");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Vehicle vehicle = new Car();  
  
        vehicle.start();  
        vehicle.stop();  
    }  
}
```

Output

Car started
Vehicle stopped

2. Static Methods

Interfaces can contain static utility methods.

Example

```
interface MathUtility {  
  
    static int square(int x) {  
        return x * x;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println(MathUtility.square(5));  
    }  
}
```

Output

25

Built-in Functional Interfaces in Java 8

Java 8 provides many functional interfaces in the java.util.function package.

Interface	Abstract Method	Purpose
Predicate<T>	test()	Returns boolean
Function<T,R>	apply()	Converts one type to another
Consumer<T>	accept()	Consumes data, returns nothing
Supplier<T>	get()	Produces data
UnaryOperator<T>	apply()	Same input and output type
BinaryOperator<T>	apply()	Operates on two values of the same type

BiFunction<T,U,R>	apply()	Two inputs, one output
BiConsumer<T,U>	accept()	Consumes two values
BiPredicate<T,U>	test()	Tests two values

Comparison

Feature	Normal Interface	Functional Interface
Abstract methods	Multiple allowed	Exactly one
Lambda expression	Not supported	Supported
Method reference	Not supported	Supported
Default methods	Yes (Java 8)	Yes
Static methods	Yes (Java 8)	Yes
@FunctionalInterface annotation	Not used	Recommended
Primary use	Define contracts	Functional programming and lambdas

Summary

Java 8 interfaces can be broadly classified into:

1. **Normal (Regular) Interface** – Contains one or more abstract methods and defines a contract for implementing classes.
2. **Functional Interface (SAM Interface)** – Contains exactly one abstract method and is designed for lambda expressions, method references, and the Stream API.

In addition, Java 8 enhanced interfaces by allowing **default methods** and **static methods**, making interfaces more powerful and enabling backward-compatible API evolution.

In Java 8, the four most commonly used functional interfaces are:

1. **Function<T, R>**
2. **Predicate**
3. **Consumer**
4. **Supplier**

These are part of the `java.util.function` package and are the foundation of the **Stream API**, **Lambda Expressions**, and **Functional Programming**.

How to Choose the Correct Functional Interface

Ask these questions in order:

Do I need to produce a value without any input?

Yes

Supplier

No

Do I need to perform an action only?

Yes

Consumer

No

Do I need a True/False result?

Yes

Predicate

No

Do I need to convert one value into another?

Yes

Function

1. **Function<T, R>**

Definition

A **Function** accepts **one input** and returns **one output**.

Input -----> Processing -----> Output

Method

R apply(T t);

- **T** = Input type
 - **R** = Return type
-

Algorithm

Start

Receive an input

Process the input

Return a new value

End

Example 1: Square a Number

```
Function<Integer, Integer> square = n -> n * n;
```

```
System.out.println(square.apply(5));
```

Output

25

Example 2: Convert String to Uppercase

```
Function<String, String> upper = String::toUpperCase;
```

```
System.out.println(upper.apply("java"));
```

Output

JAVA

When to Use Function

Use a Function when you need to:

- Convert one object into another.
- Transform data.
- Map one type to another.

Examples:

- Employee → EmployeeDTO
 - String → Integer
 - Celsius → Fahrenheit
 - Name → Uppercase Name
-

2. Predicate

Definition

A **Predicate** accepts one input and returns **true** or **false**.

Input -----> Condition -----> true / false

Method

```
boolean test(T t);
```

Algorithm

Receive input

Check condition

If condition satisfied

 return true

Else

 return false

Example 1: Check Even Number

```
Predicate<Integer> even = n -> n % 2 == 0;
```

```
System.out.println(even.test(10));  
System.out.println(even.test(7));
```

Output

```
true  
false
```

Example 2: Check Adult

```
Predicate<Integer> adult = age -> age >= 18;
```

When to Use Predicate

Use a Predicate for filtering or validation.

Examples:

- Age >= 18
 - Salary > 50000
 - Email is valid
 - Name starts with "A"
 - Employee is active
-

3. Consumer

Definition

A **Consumer** accepts one input but **returns nothing**.

It performs an action (a side effect).

Input -----> Perform Action

No Return Value

Method

```
void accept(T t);
```

Algorithm

Receive input

Perform required operation

Return nothing

Example 1: Print Name

```
Consumer<String> print = System.out::println;  
print.accept("Java");
```

Output

Java

Example 2: Save Employee

```
Consumer<Employee> save = employee -> repository.save(employee);
```

When to Use Consumer

Use a Consumer when you need to:

- Print data.
- Save data.
- Send emails.
- Log messages.
- Update records.

Examples:

- Display employee details.
 - Write to a file.
 - Store data in a database.
 - Send notifications.
-

4. Supplier

Definition

A **Supplier** returns a value but **takes no input**.

No Input -----> Generate Value

Method

T get();

Algorithm

No input

Generate data

Return data

Example 1: Random Number

```
Supplier<Integer> random =  
    () -> new Random().nextInt(100);
```

```
System.out.println(random.get());
```

Example 2: Current Date

```
Supplier<LocalDate> today = LocalDate::now;
```

```
System.out.println(today.get());
```

When to Use Supplier

Use a Supplier when:

- Generating IDs.
 - Creating objects.
 - Returning the current date/time.
 - Lazy loading.
 - Providing default values.
-

Real-World Example: Employee Processing

Suppose you have a list of employees.

Employee

↓

Predicate

Is salary > 50,000?

↓
Function
Convert Employee → EmployeeDTO
↓
Consumer
Print DTO

Example:

```
employees.stream()  
    .filter(emp -> emp.getSalary() > 50000) // Predicate  
    .map(emp -> new EmployeeDTO(emp))      // Function  
    .forEach(System.out::println);         // Consumer
```

Stream API Mapping

Stream Method	Functional Interface	Purpose
filter()	Predicate	Filter elements based on a condition
map()	Function	Transform each element
forEach()	Consumer	Perform an action on each element
generate()	Supplier	Generate elements

Comparison Table

Functional Interface	Input	Output	Main Method	Typical Use
Function<T,R>	One	One	apply()	Convert or transform data
Predicate<T>	One	boolean	test()	Validate or filter data
Consumer<T>	One	None	accept()	Perform an action
Supplier<T>	None	One	get()	Generate or provide data

Decision Guide

Requirement	Use
Convert one value into another	Function
Check a condition and return true/false	Predicate
Perform an action without returning a value	Consumer
Produce a value without taking any input	Supplier

Interview Tips

- **Function:** "Transforms one object into another."
- **Predicate:** "Checks a condition and returns a boolean."
- **Consumer:** "Consumes data and performs an action without returning anything."
- **Supplier:** "Produces data without requiring any input."

These four interfaces form the core of Java 8's functional programming model and are used extensively with lambda expressions, method references, and the Stream API to write concise, readable, and reusable code.

Short Answer: Yes, in Java 8, any interface that has exactly one abstract method is considered a functional interface, whether or not it is annotated with @FunctionalInterface.

However, there are a few important rules and exceptions to understand.

Definition of a Functional Interface

A **Functional Interface** is an interface that contains **exactly one abstract method (SAM - Single Abstract Method)**.

It may also contain:

- Any number of **default methods**
- Any number of **static methods**
- Methods inherited from Object (such as toString(), equals(), and hashCode()), which do **not** count as abstract methods

Because there is only one abstract method, Java can determine which method a lambda expression implements.

Algorithm to Determine Whether an Interface Is Functional

Start

Count the abstract methods in the interface.

If the count is exactly 1

It is a Functional Interface.

Else

It is NOT a Functional Interface.

Ignore:

- default methods
- static methods
- Object methods

End

Example 1: Functional Interface (Without Annotation)

```
interface Calculator {  
  
    int add(int a, int b);  
}
```

This interface has:

- 1 abstract method ✓

Therefore, it is a **functional interface**, even though `@FunctionalInterface` is not used.

You can write:

```
Calculator c = (a, b) -> a + b;
```

```
System.out.println(c.add(10, 20));
```

Output

30

Example 2: Functional Interface (With Annotation)

```
@FunctionalInterface
interface Greeting {

    void sayHello();
}
```

The annotation is **optional**, but recommended because it lets the compiler verify that the interface remains functional.

```
Greeting g = () -> System.out.println("Hello!");

g.sayHello();
```

Output

Hello!

Example 3: One Abstract Method + Default Method

```
@FunctionalInterface
interface Vehicle {

    void start();

    default void stop() {
        System.out.println("Stopped");
    }
}
```

Abstract methods:

- start() ✓

Default methods:

- stop() ✗(does not count)

This is still a **functional interface**.

Example 4: One Abstract Method + Static Method

```
@FunctionalInterface
interface MathOperation {
```

```
int square(int n);

static void info() {
    System.out.println("Utility Method");
}
}
```

Only square() is abstract, so this is also a functional interface.

Example 5: Not a Functional Interface

```
interface Animal {

    void eat();

    void sleep();
}
```

Abstract methods:

- eat()
- sleep()

Since there are **two abstract methods**, this is **not** a functional interface.

Trying to assign a lambda expression:

```
Animal a = () -> System.out.println("Eating");
```

results in a compilation error because Java does not know which abstract method the lambda should implement.

Example 6: Methods from Object

```
@FunctionalInterface
interface Person {

    void display();

    String toString();
}
```

Although toString() appears in the interface, it is already defined in Object, so it **does not count** as an additional abstract method.

The interface still has only one abstract method (`display()`), so it is a functional interface.

Why Use `@FunctionalInterface`?

The annotation is **not required**, but it helps catch mistakes.

Correct:

```
@FunctionalInterface
interface Test {

    void show();
}
```

Incorrect:

```
@FunctionalInterface
interface Test {

    void show();

    void print();
}
```

Compiler error:

```
Unexpected @FunctionalInterface annotation
Test is not a functional interface
Multiple non-overriding abstract methods found
```

The annotation acts as a compile-time safety check.

Real-World Example

Suppose you need a salary calculation strategy.

```
@FunctionalInterface
interface SalaryCalculator {

    double calculate(double basicSalary);
}
```

Different implementations can be written using lambdas:

SalaryCalculator yearly = salary -> salary * 12;

SalaryCalculator bonus = salary -> salary * 1.20;

Each lambda provides the implementation of the single abstract method calculate().

Interview Points

- A functional interface has **exactly one abstract method**.
- `@FunctionalInterface` is **optional**, not mandatory.
- Default methods and static methods **do not count** toward the abstract method count.
- Methods inherited from `Object` also **do not count**.
- Functional interfaces are the basis for **lambda expressions**, **method references**, and much of the **Stream API**.

Summary Table

Interface	Abstract Methods	Functional Interface?
<code>interface A { void m(); }</code>	1	✓ Yes
<code>interface B { void m1(); void m2(); }</code>	2	✗ No
<code>interface C { void m(); default void d() }</code>	1	✓ Yes
<code>interface D { void m(); static void s() }</code>	1	✓ Yes
<code>@FunctionalInterface interface E { void m(); }</code>	1	✓ Yes
<code>@FunctionalInterface interface F { void m1(); void m2(); }</code>	2	✗ Compilation error

In summary, **every interface with exactly one abstract method is a functional interface in Java 8**, regardless of whether it is annotated with `@FunctionalInterface`. The annotation is recommended because it allows the compiler to enforce the rule and prevent accidental changes that would make the interface non-functional.

In **Java 8**, the **:: operator** is called the **Method Reference Operator**.

A **method reference** is a shorthand way of writing a lambda expression when the lambda simply calls an existing method. It improves readability and makes the code more concise.

Definition

A **method reference** is a reference to an existing method or constructor using the **:: operator**.

Instead of writing:

```
x -> x.toUpperCase()
```

you can write:

```
String::toUpperCase
```

Both do exactly the same work.

Why was **::** introduced?

Before Java 8, you often had to create anonymous classes.

With Java 8, lambdas reduced boilerplate.

When a lambda only calls an existing method, **::** makes it even shorter and easier to read.

Example:

Lambda:

```
list.forEach(name -> System.out.println(name));
```

Method Reference:

```
list.forEach(System.out::println);
```

Both produce the same output.

Internal Working (Algorithm)

Suppose you write:

```
list.forEach(System.out::println);
```

Internally, Java treats it approximately like:

```
Consumer<String> consumer = s -> System.out.println(s);
```

```
list.forEach(consumer);
```

Algorithm:

Start

Compiler sees the :: operator.

Find the target functional interface.

Check its abstract method signature.

Find a matching existing method.

Create an implementation of the functional interface.

Invoke that method whenever the functional interface is called.

End

The actual implementation uses the JVM's **invokedynamic** instruction and **LambdaMetafactory**, so it is more efficient than generating anonymous inner classes.

Types of Method References

There are **four types** of method references.

1. Reference to a Static Method

Syntax

```
ClassName::staticMethod
```

Example:

```
import java.util.function.Function;
```

```
public class Demo {
```

```
    public static int square(int x) {  
        return x * x;  
    }
```

```

    }

    public static void main(String[] args) {

        Function<Integer, Integer> function = Demo::square;

        System.out.println(function.apply(5));
    }
}

```

Output:

25

Equivalent lambda:

```
Function<Integer, Integer> function = x -> Demo.square(x);
```

2. Reference to an Instance Method of a Particular Object

Syntax

objectReference::instanceMethod

Example:

```

public class Printer {

    public void print(String message) {
        System.out.println(message);
    }

    public static void main(String[] args) {

        Printer printer = new Printer();

        Consumer<String> consumer = printer::print;

        consumer.accept("Hello Java");
    }
}

```

Output:

Hello Java

Equivalent lambda:

```
Consumer<String> consumer = s -> printer.print(s);
```

3. Reference to an Instance Method of an Arbitrary Object

Syntax

ClassName::instanceMethod

Example:

```
List<String> names =  
    Arrays.asList("java", "spring", "sql");  
  
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Output:

```
JAVA  
SPRING  
SQL
```

Equivalent lambda:

```
.map(s -> s.toUpperCase())
```

Here, each element of the stream becomes the receiver of the `toUpperCase()` method.

4. Reference to a Constructor

Syntax

ClassName::new

Example:

```
Supplier<ArrayList<String>> supplier = ArrayList::new;  
  
ArrayList<String> list = supplier.get();
```

Equivalent lambda:

```
Supplier<ArrayList<String>> supplier =  
    () -> new ArrayList<>();
```

How Java Chooses the Method

Suppose:

```
Function<String, Integer> function = Integer::parseInt;
```

The compiler checks:

Function

```
R apply(T t);
```

becomes

```
Integer apply(String value);
```

Then it searches for:

```
Integer.parseInt(String)
```

Since the signatures match, the method reference is valid.

Common Functional Interfaces with Method References

Function

```
Function<String, Integer> length = String::length;
```

```
System.out.println(length.apply("Java"));
```

Output:

4

Equivalent lambda:

```
s -> s.length()
```

Predicate

```
Predicate<String> predicate = String::isEmpty;
```

```
System.out.println(predicate.test(""));
```

Output:

true

Equivalent lambda:

```
s -> s.isEmpty()
```

Consumer

```
Consumer<String> consumer = System.out::println;
```

```
consumer.accept("Hello");
```

Equivalent lambda:

```
s -> System.out.println(s);
```

Supplier

```
Supplier<Date> supplier = Date::new;
```

```
System.out.println(supplier.get());
```

Equivalent lambda:

```
() -> new Date();
```

Stream API Examples

Sorting

Lambda:

```
list.sort((a, b) -> a.compareTo(b));
```

Method Reference:

```
list.sort(String::compareTo);
```

Printing

Lambda:

```
list.forEach(name -> System.out.println(name));
```

Method Reference:

```
list.forEach(System.out::println);
```

Mapping

Lambda:

```
list.stream()  
    .map(s -> s.toUpperCase());
```

Method Reference:

```
list.stream()  
    .map(String::toUpperCase);
```

Lambda vs Method Reference

Lambda Expression	Method Reference
<code>x -> Math.abs(x)</code>	<code>Math::abs</code>
<code>x -> x.length()</code>	<code>String::length</code>
<code>x -> System.out.println(x)</code>	<code>System.out::println</code>
<code>() -> new ArrayList<>()</code>	<code>ArrayList::new</code>
<code>s -> Integer.parseInt(s)</code>	<code>Integer::parseInt</code>

Advantages

- Less code to write.
 - Improves readability.
 - Reuses existing methods instead of creating new lambda bodies.
 - Integrates naturally with the Stream API.
 - Uses the same efficient runtime mechanism as lambda expressions (invokedynamic).
-

When to Use ::

Use a method reference when your lambda **only calls an existing method** without adding extra logic.

Examples:

```
.map(String::trim)
```

```
.map(String::toUpperCase)
```

```
.filter(String::isEmpty)
```

```
.forEach(System.out::println)
```

```
Supplier<List<String>> supplier = ArrayList::new;
```

If your lambda performs additional operations, use a lambda instead.

For example:

```
name -> name.trim().toUpperCase()
```

cannot be replaced by a single method reference because it chains two method calls.

Interview Summary

- `::` is the **method reference operator** introduced in Java 8.
- It is a shorthand for lambda expressions that simply invoke an existing method or constructor.
- It works **only with functional interfaces** because the compiler maps the referenced method to the interface's single abstract method.
- There are **four types** of method references:
 1. Static method reference (`ClassName::staticMethod`)
 2. Instance method reference of a particular object (`object::instanceMethod`)
 3. Instance method reference of an arbitrary object of a particular type (`ClassName::instanceMethod`)
 4. Constructor reference (`ClassName::new`)
- Internally, method references are implemented using the same JVM mechanism as lambdas (`invokedynamic` and `LambdaMetafactory`), making them both concise and efficient.

`ArrayList::new` is called a **constructor reference** in Java 8.

It is a special type of **method reference** (`::`) that refers to a class constructor instead of a normal method.

It is commonly used with **functional interfaces** such as `Supplier`, `Function`, and `BiFunction`.

Definition

A constructor reference is a shorthand for creating new objects.

Instead of writing:

```
() -> new ArrayList<>()
```

you can write:

```
ArrayList::new
```

Both create a new `ArrayList` object.

Syntax

```
ClassName::new
```

Examples:

```
ArrayList::new
```

```
HashMap::new
```

```
String::new
```

```
Employee::new
```

Internal Working

Consider the code:

```
Supplier<ArrayList<String>> supplier = ArrayList::new;
```

The compiler sees:

Supplier<T>

whose abstract method is:

T get();

The compiler searches for a constructor that:

- Takes **no arguments**
- Returns an ArrayList

It finds:

```
public ArrayList()
```

and internally treats it approximately as:

```
Supplier<ArrayList<String>> supplier =  
    () -> new ArrayList<>();
```

When you execute:

```
supplier.get();
```

Java internally executes:

```
new ArrayList<>();
```

Internal Algorithm

Start

Compiler sees ArrayList::new

↓

Find the target functional interface

↓

Read its abstract method

Supplier:
ArrayList get()

↓

Search ArrayList constructors

↓

Find constructor:
ArrayList()

↓

Generate implementation

↓

When get() is called

↓

Execute

new ArrayList()

↓

Return newly created object

End

Example 1: Supplier

```
import java.util.ArrayList;
import java.util.function.Supplier;

public class Demo {

    public static void main(String[] args) {

        Supplier<ArrayList<String>> supplier =
            ArrayList::new;

        ArrayList<String> list = supplier.get();

        list.add("Java");
        list.add("Spring");

        System.out.println(list);
    }
}
```

Output

[Java, Spring]

Equivalent Lambda

```
Supplier<ArrayList<String>> supplier =  
    () -> new ArrayList<>();
```

Example 2: Constructor with One Parameter

Suppose:

```
class Employee {  
  
    private String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String toString() {  
        return name;  
    }  
}
```

Use:

```
Function<String, Employee> function =  
    Employee::new;  
  
Employee employee = function.apply("Anurag");  
  
System.out.println(employee);
```

Equivalent Lambda

```
Function<String, Employee> function =  
    name -> new Employee(name);
```

Internal Working

Compiler checks:

Function<T,R>

R apply(T value)

becomes

Employee apply(String)

It searches constructors:

Employee(String)

Signature matches.

Therefore,

Employee::new

is valid.

Example 3: Two-Parameter Constructor

```
class Employee {  
  
    String name;  
    int age;  
  
    Employee(String name, int age) {  
  
        this.name = name;  
        this.age = age;  
    }  
}
```

Use

```
BiFunction<String, Integer, Employee> function =  
    Employee::new;  
  
Employee employee = function.apply("Anurag", 30);
```

Equivalent Lambda

```
BiFunction<String,Integer,Employee> function =  
    (name, age) ->  
        new Employee(name, age);
```

Example 4: Stream API

Without Constructor Reference

```
Supplier<List<String>> supplier =  
    () -> new ArrayList<>();
```

With Constructor Reference

```
Supplier<List<String>> supplier =  
    ArrayList::new;
```

Cleaner and easier to read.

Multiple Constructors

Suppose:

```
class Student {  
  
    Student() {  
    }  
  
    Student(String name) {  
    }  
  
    Student(String name, int age) {  
    }  
}
```

Java selects the constructor based on the functional interface.

Supplier

```
Supplier<Student> s =  
    Student::new;
```

Calls

```
Student()
```

Function

```
Function<String, Student> f =  
    Student::new;
```

Calls

Student(String)

BiFunction

```
BiFunction<String,Integer,Student> b =  
    Student::new;
```

Calls

Student(String,int)

How the Compiler Chooses the Constructor

Suppose:

```
BiFunction<String,Integer,Employee>
```

The compiler checks

```
Employee apply(String,Integer)
```

Then searches for

```
Employee(String,Integer)
```

If found

✓Valid

Otherwise

✗Compilation Error

Relationship with Functional Interfaces

Functional Interface	Constructor Selected	Example
Supplier<T>	No-argument constructor	ArrayList::new
Function<T,R>	One-argument constructor	Employee::new

BiFunction<T,U,R>	Two-argument constructor	Employee::new
Custom Functional Interface	Matching constructor	Depends on the abstract method signature

What Happens at Runtime?

When you write:

```
Supplier<ArrayList<String>> supplier =
    ArrayList::new;
```

The JVM does **not** create an anonymous inner class.

Instead, Java 8 uses:

- invokedynamic
- LambdaMetafactory

to create an efficient implementation of the functional interface that invokes the appropriate constructor when `get()` is called.

Conceptually, it behaves like:

```
class SupplierImpl
    implements Supplier<ArrayList<String>> {

    @Override
    public ArrayList<String> get() {

        return new ArrayList<>();
    }
}
```

The JVM generates this behavior dynamically instead of you writing the class yourself.

Constructor Reference vs Lambda

Lambda	Constructor Reference
<code>() -> new ArrayList<>()</code>	<code>ArrayList::new</code>
<code>name -> new Employee(name)</code>	<code>Employee::new</code>
<code>(n,a) -> new Employee(n,a)</code>	<code>Employee::new</code>

--	--

Advantages

- Less code to write.
 - Improves readability.
 - Eliminates unnecessary lambda expressions.
 - Uses efficient JVM implementation (invokedynamic).
 - Integrates naturally with the Stream API and functional interfaces.
-

When to Use ArrayList::new

Use a constructor reference when:

- You need to create objects through a functional interface.
- The lambda only calls a constructor and performs no additional logic.
- You are working with Supplier, Function, BiFunction, or custom functional interfaces.

Examples include:

- Creating collections (ArrayList::new, HashMap::new).
 - Creating domain objects (Employee::new, Student::new).
 - Object factories.
 - Stream collectors and APIs that accept suppliers.
-

Interview Summary

- ArrayList::new is a **constructor reference** introduced in Java 8.
- It is shorthand for `() -> new ArrayList<>()`.
- It can be used only where a **functional interface** is expected.
- The compiler matches the constructor to the functional interface's single abstract method signature.
- At runtime, Java uses **invokedynamic** and **LambdaMetafactory** to efficiently create the implementation, avoiding the overhead of anonymous inner classes.

Runnable vs Callable in Java

Runnable and Callable<V> are two interfaces used to execute tasks concurrently (in separate threads). The key difference is that **Runnable performs a task without returning a result**, whereas **Callable<V> performs a task and returns a value (or throws a checked exception)**.

What is Runnable?

Definition

Runnable is a **functional interface** introduced in **Java 1.0**.

It represents a task that can be executed by a thread.

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

Characteristics

- Introduced in Java 1.0
 - Functional Interface
 - Single abstract method: run()
 - Returns **void**
 - Cannot throw checked exceptions directly
 - Used with Thread and ExecutorService
-

Example 1: Using Thread

```
public class RunnableExample {

    public static void main(String[] args) {

        Runnable task = () -> {
            System.out.println("Task is running...");
        };

        Thread thread = new Thread(task);

        thread.start();
    }
}
```

Output

Task is running...

Internal Working

Runnable Object

↓

Thread

↓

start()

↓

New Thread Created

↓

run()

↓

Task Executes

Example 2: Multiple Threads

```
Runnable task = () -> {
```

```
    for(int i=1;i<=5;i++)
        System.out.println(i);
};
```

```
new Thread(task).start();
```

What is Callable?

Definition

Callable<V> is a **functional interface** introduced in **Java 5**.

It represents a task that **returns a result**.

```
@FunctionalInterface
public interface Callable<V> {

    V call() throws Exception;

}
```

Here,

V = Return Type

Example

Callable<Integer>

returns Integer

Callable<String>

returns String

Callable<Employee>

returns Employee

Characteristics

- Introduced in Java 5
 - Generic Interface
 - Returns value
 - Can throw checked exceptions
 - Executed using ExecutorService
 - Returns a Future<V>
-

Example

```
import java.util.concurrent.*;

public class CallableExample {

    public static void main(String[] args) throws Exception {

        ExecutorService executor =
            Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> {
```

```
        return 100;
    };

    Future<Integer> future =
        executor.submit(task);

    System.out.println(future.get());

    executor.shutdown();
}
}
```

Output

100

Internal Working

Callable

↓

ExecutorService

↓

submit()

↓

Worker Thread

↓

call()

↓

Result

↓

Future

↓

future.get()

Future

When a Callable executes,

it does **not** immediately return the result.

Instead it returns

Future<V>

Example

```
Future<Integer> future =  
executor.submit(task);
```

Later

```
Integer result =  
future.get();
```

get() waits until the task completes.

Example: Sum of Numbers

```
Callable<Integer> task = () -> {
```

```
    int sum = 0;
```

```
    for(int i=1;i<=100;i++)  
        sum += i;
```

```
    return sum;
```

```
};
```

Output

5050

Runnable Equivalent

Runnable cannot return

```
Runnable task = () -> {  
    return 100; // Compilation Error  
};
```

Error

Cannot return a value

Checked Exceptions

Runnable

```
Runnable task = () -> {  
    throw new IOException();  
};
```

Compilation Error

Unhandled checked exception

Because

run()

does not declare

throws Exception

Callable

```
Callable<Integer> task = () -> {  
    throw new IOException();  
};
```

Perfectly valid because

call()

throws Exception

Lambda Expressions

Runnable

```
Runnable r =  
() -> System.out.println("Java");
```

Callable

```
Callable<String> c =  
() -> "Java";
```

Thread vs ExecutorService

Runnable with Thread

```
Runnable r = () -> {  
    System.out.println("Hello");  
};
```

```
Thread t = new Thread(r);
```

```
t.start();
```

Callable with ExecutorService

```
ExecutorService executor =  
Executors.newSingleThreadExecutor();
```

```
Callable<String> c =  
() -> "Hello";
```

```
Future<String> future =  
executor.submit(c);
```

```
System.out.println(future.get());
```

Real-world Example

Suppose you are downloading a file.

Runnable

Download file

↓

Done

(No result)

Callable

Download file

↓

Return downloaded file size

↓

Caller receives size

Another Example

Find factorial

Runnable

Runnable r = () -> {

 int fact = 1;

 for(int i=1;i<=5;i++)
 fact *= i;

 System.out.println(fact);

};

Output

120

Value is only printed.

Callable

Callable<Integer> c = () -> {

```
int fact = 1;

for(int i=1;i<=5;i++)
    fact *= i;

return fact;

};
```

Output

120

The value can now be stored and reused.

Execution Flow Comparison

Runnable

Runnable

↓

Thread

↓

run()

↓

Execute Task

↓

Finish

Callable

Callable

↓

ExecutorService

↓

submit()

↓

call()

↓

Return Result

↓

Future

↓

get()

Common Use Cases

Runnable

- Logging
 - Sending notifications
 - Printing reports
 - Background cleanup
 - Cache refresh
 - Fire-and-forget tasks
-

Callable

- Database queries
 - REST API calls
 - File downloads
 - Mathematical calculations
 - Image processing
 - Report generation
 - Returning computed values
-

Advantages

Runnable

- Simple
 - Lightweight
 - Easy to use
 - Compatible with Thread
 - Good for tasks without results
-

Callable

- Returns results
 - Supports checked exceptions
 - Works with Future
 - Better for asynchronous computations
 - Generic return type
-

Runnable vs Callable

Feature	Runnable	Callable
Package	java.lang	java.util.concurrent
Introduced	Java 1.0	Java 5
Functional Interface	Yes	Yes
Abstract Method	run()	call()
Return Type	void	V (Generic)
Returns Value	No	Yes
Throws Checked Exception	No	Yes (throws Exception)
Generic	No	Yes
Used With	Thread, ExecutorService	ExecutorService
Submission Method	execute() or submit()	submit()
Return Object	None	Future<V>
Best For	Fire-and-forget tasks	Tasks that produce a result

Summary

- **Runnable** is intended for tasks that perform work but **do not return a result**. Its `run()` method returns void and cannot declare checked exceptions. It can be executed using a Thread or an ExecutorService.
- **Callable<V>** is designed for tasks that **produce a result**. Its `call()` method returns a value of type V and may throw checked exceptions. It is executed through an ExecutorService, and the result is obtained via a `Future<V>`.
- Choose **Runnable** for simple background or fire-and-forget operations. Choose **Callable<V>** when you need to retrieve a computed value, propagate checked exceptions, or coordinate asynchronous computations.